🔐 KeyPrint Guard & Arena: 타이핑 생체 지문(Keystroke Dynamics) 임베딩 기반 실시간 해커 침입 차단(1:1) 및 타자연습 인물 자동 분류(1:N) 시스템

> **"비밀번호가 유출되어도 해커는 당신의 손가락 근육 리듬을 복제할 수 없으며, 똑같은 타자연습 문장을 쳐도 AI는 누가 타이핑했는지 알아냅니다."** 키보드의 미세 누름 지속 시간(`Hold`), 키 간 전환 지연(`DD`), 비행 시간(`UD`) 및 연타 롤오버(`UD < 0`) 파형으로부터 **64차원 초구면 생체 지문 임베딩(64-D Hyperspherical Biometric Embedding)**을 추출하여, 브라우저 내부(ONNX Runtime Web WASM)에서 서버 전송 없이 **0.7ms** 만에 **① 유출 암호 해커 침입 차단(1:1 검증)**과 **② 타자연습 다중 인물 자동 식별(1:N 분류)**을 수행하는 End-to-End 머신러닝 보안·생체인식 시스템입니다.

---

0. 프로젝트 정보

항목	내용
과목명	기계학습 프로그래밍 (기말 포트폴리오 프로젝트)
학번 / 이름	`201935607` / **김대현**
프로젝트명	**KeyPrint Guard & Arena**: 키스트로크 다이나믹스 생체 임베딩 기반 실시간 해커 침입 차단(1:1) 및 타자연습 인물 자동 분류(1:N) 시스템
GitHub 저장소	https://github.com/qwertypotter6873/keyprint-guard
배포 데모 URL	https://qwertypotter6873.github.io/keyprint-guard/
데이터 유형	시계열·정형 행동 생체 신호 (`Tabular & Temporal Behavioral Biometrics`)
학습 방식	**직접 학습 (`Scratch + Deep Metric Learning`)** — 다중 코퍼스·가변 길이 시퀀스 통합 메트릭 러닝
비교 및 제안 모델	① `ScaledManhattanDetector` (통계 베이스라인) · ② `Vanilla1DCNN` (딥러닝 베이스라인) · ③ **`KeyPrintNet`** (제안 모델, 최종 채택)
주요 기술 스택	`Python 3.12` , `PyTorch` , `Scikit-Learn` , `ONNX (Opset 17)` , `ONNX Runtime Web (WASM SIMD)` , `Vite` , `Vanilla ES2022 JS` , `GitHub Actions / Pages`

---

1. 문제 정의

1.1 해결하려는 실제 문제 배경

1. **유출 암호 기반 계정 탈취(Credential Stuffing & Phishing)의 구조적 취약성**: 기존 텍스트 비밀번호 인증은 문자열 일치(`typed == stored`)만을 검사하는 이진 논리에 의존합니다. 피싱, 키로거, 다크웹 DB 유출, 어깨너머 훔쳐보기(Shoulder Surfing)로 비밀번호 문자열이 노출되는 순간, 숙련된 인간 해커나 자동화 매크로 스크립트(`AutoHotkey` ,

`Selenium` 등)의 계정 침입을 전혀 방어할 수 없습니다. 또한 OTP·생체센서(지문·안면)는 추가 하드웨어나 별도 기기 조작 마찰을 유발합니다.

2. **공유 텍스트·타자연습 환경에서의 무자각 행위자 식별("Who Is Typing?")**: 여러 사용자가 동일한 연습 문장이나 표준 명령어를 타이핑할 때, 입력된 결과 문자열은 100% 동일하지만 사람마다 손가락 길이, 건반 간 도약 습관, 두 키를 겹쳐 누르는 **연타 롤오버(`UD < 0`)** 비율이 고유한 신경근육 기억(Neuromuscular Memory)을 형성합니다. 이를 정교한 임베딩 공간으로 사상할 수 있다면, **타자연습을 몇 차례 수행해 두는 것만으로 나중에 똑같은 문장(혹은 새로운 문장)을 쳤을 때 지금 키보드 앞에 앉은 사람이 누구인지 자동으로 분류**할 수 있습니다.

1.2 핵심 과업 정의 및 입·출력 명세

본 프로젝트는 단일 통합 신경망 인코더(`KeyPrintNet`)로 두 가지 실용적 행동 생체인식 과업을 동시에 해결합니다.

구분	[모드 1] 1:1 유출 암호 해커 침입 차단 (Open-Set Verification)	[모드 2] 1:N 타자연습 인물 자동 분류 (Multi-User Identification)
상황 가정	해커·매크로 봇이 **피해자의 정확한 비밀번호 문자열을 이미 알고 입력**한 위기 상황	`N` 명의 사용자가 타자연습 문장을 각 3~5회씩 등록해 둔 뒤, 누군가 나와 문장을 1회 타이핑한 상황
모델 입력 (Input)	`M` 개 연속 키 전이(Digraph)로부터 추출한 **9채널 생체역학 시퀀스 텐서** `X ∈ ℝ^(9×L)` (`L = M − 1` , 가변 길이 `5 ≤ L ≤ 44`)	좌동 — 영문 에세이·표준 타자연습 문장 및 한글 속담(두벌식 물리 자모 키 이벤트)의 가변 길이 시퀀스 `X ∈ ℝ^(9×L)`
인코더 출력	단위 초구면($\|z\|_2 = 1$) 상의 **64차원 타건 생체 지문 임베딩** `z ∈ ℝ^64`	좌동 — 길이 불변 **64차원 타건 생체 지문 임베딩** `z ∈ ℝ^64`
최종 출력 (Output)	코호트 교정 생체 일치도 `S ∈ [0, 1]` , **승인 (ACCESS GRANTED) / 차단(INTRUSION BLOCKED)**, 키 구간별 Z-Score 이상치 히트맵	등록된 `N` 명 전원에 대한 **실시간 일치도 Top-1 ~ Top- N 순위표**, 사후 확률(Softmax Confidence), **미등록 외부인 (Unknown Typist) 기각 판정**

1.3 정량적 목표 설정

모델이 학습 과정에서 단 한 번도 본 적 없는 **완전 미학습 피험자(`Subject-Disjoint Unseen Test Split`)**를 대상으로 다음 목표를 설정하였습니다.

- **[과업 1 · 1:1 해커 차단]** 미학습 피험자 10인 · 3,600건 교차 세션 검증 쌍 기준 **`Test Macro F1 ≥ 0.85`** , **`ROC-AUC ≥ 0.92`** , **`EER(동일오류율) ≤ 15.0%`** 달성
- **[과업 2 · 1:N 인물 분류]** 완전히 동일한 문구를 쳤을 때 미학습 10인 분류 **`Top-1 ≥ 40%` (무작위 10% 대비 4배↑),** **`Top-3 ≥ 88%`** 및 영문 타자연습 문장(25자) 미학습 18인 분류 **`Top-1 ≥ 55%` (무작위 5.56% 대비 10배↑),** **`Top-3 ≥ 88%`** 달성
- **[실시간 웹 서비스]** 외부 서버 통신 없는 브라우저 로컬 WebAssembly 추론 **단일 타건 지연 시간 `≤ 5.0 ms`** , 모델 가중치 크기 **`≤ 1.0 MB`** 달성

---

2. 데이터

고정 비밀번호 해커 차단(`1:1`)과 자연어 타자연습 문장 인물 식별(`1:N`)을 모두 실제 인간 타건 데이터로 학습·검증하기 위해, 학술적으로 검증된 **이중 공개 벤치마크 코퍼스(Dual Real-World Keystroke Corpora)**를 수집·구축하였습니다.

2.1 수집 데이터셋 명세 및 출처

데이터셋 구분	출처 및 라이선스	수집 프로토콜 및 텍스트 특성	원시 규모	정제 후 유효 규모	결측치 및 클래스 분포
① CMU Keystroke Dynamics Benchmark (`DSL-StrongPasswordData.csv`)	Killourhy & Maxion, *IEEE/IFIP DSN 2009* (CMU 공식 아카이브, 학술 공개)	**51명**의 피험자가 최소 1일 이상 간격을 둔 **8개 독립 세션**에서 강건 암호 `.tie5Roanl` + `Enter` (11키, `L=10` 전이)를 세션당 50회씩 반복 입력	`20,400` 행 × 34열 (`4.5 MB`)	**`20,400` 시퀀스** (51명 × `400`회)	결측치 **`0`건**, 모든 피험자가 정확히 **400건씩 보유한 완전 균형 분포**
② KeyRecs Multi-Session Transcription Dataset (`KeyRecs-free-text.csv`, `fixed-text.csv`)	Zenodo DOI `10.5281/zenodo.7886742` (*CC-BY 4.0 오픈 라이선스*)	**99명** 참가자가 **2개 독립 세션**에서 영문 문학 타자연습(세션 1: Proust 소설 필사, 세션 2: Murakami 수필 필사) 및 고정 문자열을 자연 타이핑	원시 키 이벤트 **`562,583` 행** (`24.0 MB`) + 고정 문자열 `19,772` 행	2세션 모두 보유한 **96명**의 100% 무오타 **`25`자 (`L=24`) 문장 시퀀스 `12,848`건** + 고정 문자열 `13,588`건	파일 말미 단일 `NaN` 행 외 결측 제거 완료, 참가자당 평균 `133.8`개 문장 시퀀스

2.2 키스트로크 생체 신호 구조 및 탐색적 데이터 분석 (EDA)

키 `k_t`에서 다음 키 `k_{t+1}`로 이어지는 각 디그래프(Digraph) 전이 구간마다 세 가지 물리적 타이밍 원시 변수가 측정됩니다.

- **키 홀드 시간 (`Hold`, `H_t = U_t − D_t`)**: 키 `k_t`를 누른 시점(`D_t`)부터 뗀 시점(`U_t`)까지의 지속 시간 (손가락 누름 압력·탄성 습관 반영)
- **키 간 눌림 간격 (`Down-Down`, `DD_t = D_{t+1} − D_t`)**: 현재 키를 누른 뒤 다음 키를 누르기까지의 리듬 간격 (타이핑 템포 및 손가락 배치 거리 반영)
- **키 간 비행 시간 (`Up-Down`, `UD_t = D_{t+1} − U_t = DD_t − H_t`)**: 앞 키에서 손을 뗀 후 다음 키를 누르기까지의 공중 체류 시간
  - **핵심 생체 특징 — 연타 롤오버(`Rollover`, `UD_t < 0`)**: 숙련된 타이피스트는 앞 키(`k_t`)를 완전히 떼기 **전에** 반대 손이나 인접 손가락으로 다음 키(`k_{t+1}`)를 먼저 누릅니다. CMU 데이터셋 전체 전이의 **`10.84%`**가 음수 비행시간(`UD_t < 0`)을 보이며, 사람마다 롤오버가 발생하는 글자 쌍과 깊이가 지문처럼 일관된 반면 단순 매크로 봇이나 독수리 타법 침입자는 `UD_t > 0`에 머무릅니다.
  - **세션 숙련도 속도 드리프트(Temporal Speed Drift)**: CMU 피험자들의 세션 1 평균 입력 시간 대비 세션 8 평균 입력 시간은 큰 폭으로 단축되며, 절대 타이밍(`ms`)만 비교하는 단순 거리 모델이 본인을 해커로 오차단(`FRR` 상승)하는 핵심 원인이 됩니다.

3. 데이터 전처리

원시 타건 로그의 노이즈·왜도를 제거하고 속도 변화와 문장 길이 변화에 강건한 표현을 얻기 위해 **4단계 도메인 특화 전처리 파이프라인**(`src/preprocessing.py`)을 구현하였습니다. 웹 브라우저 추론 엔진(`web/src/biometrics.js`)에도 **동일한 수식과 스케일러 통계량**을 이식하여 학습-서빙 간 괴리(Training-Serving Skew)를 원천 차단했습니다.

3.1 적용한 4가지 핵심 전처리 기법

1. **생리학적 이상치 윈저라이징(Winsorizing) 및 원시 키 이벤트 물리 타임스탬프 복원**:
   - 실험 도중 기침·딴짓·머뭇거림으로 인해 CMU 원시 데이터의 `DD.i.e` 컬럼은 최대 **`25.99초`**, 왜도(Skewness) **`73.28`**, 평균 첨도(Kurtosis) **`609.02`** 라는 극단적인 우측 꼬리 분포를 보입니다. **오직 Train 분할 데이터**의 분위수만 사용하여 생리학적 유효 범위(`H ∈ [12ms, P99.5]` , `DD ∈ [10ms, P99.5]` , `UD ∈ [P0.5, P99.5]`)로 클리핑하였습니다.
   - `KeyRecs-free-text.csv` 에서는 대문자 입력 시 `Shift` 수정자 행이 음수 `DD` (예: `W → Shift: −0.796s` , `Shift → e: +1.148s`)로 기록되어 있어 단순 행 삭제 시 타이밍이 깨집니다. 전체 누적 눌림 시각 `t_down[i] = ∑_{j < i} DD_j` 를 먼저 복원한 뒤 수정자 키를 제거하고 `Backspace` / `Delete` 등장 지점마다 시퀀스를 분절하여, 브라우저 `performance.now()` 와 **100% 물리적으로 동일한 무오타 25자(`L=24`) 타자연습 문장 시퀀스**를 복원했습니다.
2. **9채널 생체역학 디그래프 파생 피처 엔지니어링 (`4채널 → 9채널`)**:
   - 사용자의 당일 컨디션이나 숙련도에 따라 전체 타자 속도(WPM)가 빨라지더라도 **각 키 구간의 상대적 리듬 비율**은 보존된다는 생체역학적 원리에 착안하여, 시퀀스 내 평균 홀드($\bar{H}$) 및 평균 전환 간격($\bar{DD}$)으로 정규화한 5종의 파생 채널을 설계했습니다.

채널 인덱스	피처명	수식 정의	포착하는 행동 생체역학적 의미
`Ch 0`	`H_curr`	`H_t`	현재 키 `k_t` 의 물리적 누름 지속 시간
`Ch 1`	`H_next`	`H_{t+1}`	다음 키 `k_{t+1}` 의 물리적 누름 지속 시간
`Ch 2`	`DD`	`DD_t = D_{t+1} − D_t`	키 간 전환 눌림 간격 (절대 리듬 간격)
`Ch 3`	`UD`	`UD_t = DD_t − H_t`	키 간 비행 시간 (양수: 공중 이동 / 음수: 키 중첩)
`Ch 4`	**`Rel_H`** *(파생)*	`H_t / (H̄ + ε)`	**템포 불변 상대 홀드 비율** (타자 속도가 변해도 유지되는 손가락 압력 패턴)
`Ch 5`	**`Rel_DD`** *(파생)*	`DD_t / (D̄D̄ + ε)`	**템포 불변 상대 전환 리듬** (단어 내 박자 분할 구조)
`Ch 6`	**`Duty_Ratio`** *(파생)*	`clip(H_t / max(DD_t, 0.02), 0, 5)`	전환 주기 대비 건반 점유율 (스타카토 vs 레가토 타법 구분)
`Ch 7`	**`Rollover_Mag`** *(파생)*	`max(0, −UD_t)`	**비선형 연타 롤오버 깊이** (양손 교차 연타 숙련도 핵심 지문)
`Ch 8`	**`Local_Accel`** *(파생)*	`DD_t − DD_{t−1}` (`t = 0` 일 때 `0`)	연속 철자 입력 시 **순간 가속·감속 궤적**

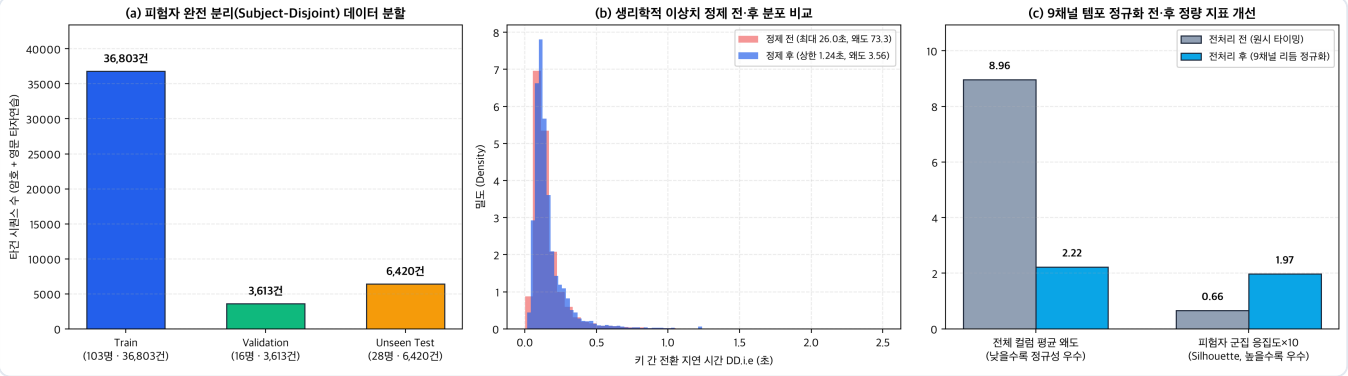
3. **피험자 완전 분리 분할(`Subject-Disjoint Split` , `seed=42`) 및 누수 없는 채널별 Z-Score 표준화**:

- 일반 행 단위 무작위 분할(`Random Row Split`)을 수행하면 동일 인물의 타건이 Train과 Test에 동시에 들어가 생체 인식 성능이 인위적으로 부풀려집니다(Data Leakage). 본 프로젝트는 **피험자 ID 단위로 Train(`103명` : CMU 35명 + KeyRecs 68명, `36,803` 건) / Validation(`16명` : CMU 6명 + KeyRecs 10명, `3,613` 건) / Unseen Test(`28명` : CMU 10명 + KeyRecs 18명, `6,420` 건)를 완전히 격리**하였고, 채널별 평균·표준편차 스케일러 역시 오직 Train 분할 시퀀스에서만 적합(`fit`)했습니다.

4. **탄성 템포 워핑(Smooth Biomechanical Tempo Warping) 데이터 증강**:
   - 학습 미니배치마다 전체 속도 배율 `α ~ U(0.80, 1.25)`와 매끄러운 사인파 가속 곡선 `w(t) = 1 + 0.06·sin(πt/L + φ)`, 미세 가우시안 지터를 물리 타이밍(`H, DD`) 단계에 적용한 뒤 `UD = DD − H` 및 9채널 파생 피처를 재계산하여 세션 간 속도 드리프트 내성을 강화했습니다.

3.2 전처리 전·후 정량 지표 비교 및 시각화

정량 비교 항목	전처리 전 (원시 타이밍 데이터)	전처리 후 (윈저라이징 + 9채널 리듬 정규화)	개선 효과 요약
전체 타이밍 컬럼 평균 왜도 (Mean Skewness ↓)	`8.9625`	`2.2239`	비대칭 꼬리 왜도 **75.2% 감소**
최대 단일 컬럼 왜도 (`DD.i.e` Max Skewness ↓)	`73.2828` (최대 `25.99초`)	`4.2503` (상한 `1.24초`)	일시정지 극단치 제거로 그래디언트 폭주 방지
전체 타이밍 컬럼 평균 첨도 (Mean Kurtosis ↓)	`609.0242`	`9.8677`	극단적 첨도 **98.4% 감소**
미학습 테스트 피험자 코사인 군집 응집도 (`Silhouette` ↑)	`0.0658` (원시 4채널)	`0.1973` (정규화 9채널)	동일 인물 타건 군집 분리도 **3.00배 향상**



---

4. 모델

동일 데이터 분할 위에서 전통 통계 모델, 일반 분류용 딥러닝 베이스라인, 그리고 본 프로젝트가 제안하는 생체 메트릭 러닝 아키텍처(`KeyPrintNet`) **총 3종의 모델을 직접 구현·학습**하여 객관적으로 비교하였습니다(`src/models.py`, `src/train_and_evaluate.py`).

4.1 비교 모델 및 제안 아키텍처 상세

구분	모델명	핵심 구조 및 작동 원리	손실 함수 / 학습 방식	가변 문장 길이(L) 지원	파라미터 / 크기
Baseline 1	**`ScaledManhattanDetector`**	Killourhy & Maxion (2009) 벤치마크 표준 통계 모델. 등록 샘플 평균 `μ_u` 및 베이즈 수축(`λ = 0.35`) 절대편차 `s_u` 기반 정규화 맨해튼 거리 산출	Train 코호트 분산 사전분포 추정 + 등록 템플릿 적합	불가 (고정 차원 벡터 전용)	통계 파라미터
Baseline 2	**`Vanilla1DCNN`**	위치 정보 없는 3층 `Conv1D(64 → 128 → 128)` + `BatchNorm` + `Global Average Pooling` → `64-D` L2 정규화 임베딩	표준 `Softmax Cross-Entropy` (`35 Epochs` , `AdamW`)	가능 (평균 풀링 시 위치 소실)	`~25K` (`103 KB`)
Proposed (최종 채택)	**`KeyPrintNet`**	**연속 상대 위치 고조파 (`Positional Harmonics`)** + **다중 스케일 잔차 SE-Conv1D(`k=3, 5, Dilated`)** + **1D-DCT 스펙트럼 피라미드 & Attentive Stats Pooling** → `64-D` 임베딩 + **AS-Norm 코호트 교정**	**도메인 분리 `Sub-Center CosFace` + `Supervised Contrastive Loss`** (`35 Epochs`)	**완벽 지원** (암호 6자 ~ 문장 45자+, ONNX 동적 축)	`147K` (**`592 KB`**)

4.2 제안 모델 `KeyPrintNet` 의 4대 핵심 설계 근거

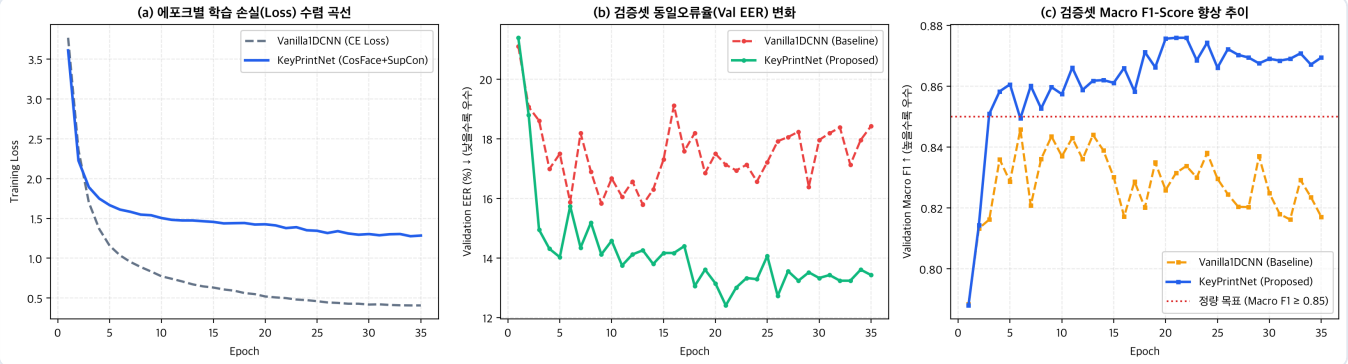
1. **연속 상대 위치 고조파 주입 (`Continuous Positional Harmonics`)**:
   - 일반 `Global Average Pooling` (`Vanilla1DCNN`)은 문장 앞부분의 타이밍과 뒷부분의 타이밍 순서를 뒤섞어도 동일한 평균값을 내어 **어느 글자 전이에서 지연·롤오버가 발생했는지 소실**됩니다. `KeyPrintNet` 은 임의의 시퀀스 길이 `L` 에 대해 정규화 위치 좌표 `τ_t = t / (L − 1) ∈ [0, 1]` 및 삼각 고조파 `[τ_t, cos(πτ_t), sin(πτ_t)]` 3채널을 입력 9채널에 결합(`12채널`)하여, 가변 길이 문장에서도 초성·중반·종결부 박자 위치를 정확히 인지합니다.
2. **다중 스케일 잔차 Squeeze-and-Excitation 블록 (`MultiScaleResSEBlock1D`)**:
   - 병렬 커널(`k=3` 인접 디그래프, `k=5` 트라이그래프·음절 리듬)과 팽창 합성곱(`dilation=2`)으로 국소·중거리 근육 협응 패턴을 동시에 포착하고, SE 채널 어텐션으로 식별력이 높은 파생 채널(`Rel_DD` , `Rollover_Mag` 등)에 가중치를 부여합니다.
3. **스펙트럼 시간 피라미드(`1D-DCT`) + 주의집중 통계 풀링 (`ASP + Spectral Pooling`)**:
   - 어텐션 가중 평균·표준편차(`160-D`)에 더해, 순수 행렬곱(`torch.bmm`) 기반 **6모드 이산 코사인 변환(`1D-DCT` , `480-D`)** 계수를 결합(`총 640-D → 64-D 투영`)하여 시퀀스 길이가 달라져도(`L = 5~32`) 문장 전체의 저주파·중주파 시간 엔벨로프(Temporal Envelope)를 보존합니다.
4. **도메인 분리 `Sub-Center CosFace` + `AS-Norm` 코호트 유사도 교정**:
   - 고정 비밀번호(`CMU` , `KeyRecs-fixed`)와 자유 필사 문장(`KeyRecs-free`)은 위치별 철자 고정 여부가 다르므로, **백본 인코더 가중치는 100% 공유하되 분류 프로토타입 헤드는 도메인별로 분리**하고 클래스당 `K = 2` 개의 서브센터를 두

어 동일 사용자의 컨디션 변동을 흡수했습니다. 추론 시에는 학습 코호트 중심점 대비 상대적 Z-점수를 계산하는 **AS-Norm(Adaptive Symmetric Score Normalization)**으로 유사 리듬 침입자(`Wolf`) 오허용을 억제합니다.

```
[입력 시퀀스 X: (B, 9, L)] ──(+)── [연속 상대 위치 고조파: τ, cos(πτ), sin(πτ) (B, 3, L)]
                                │
                  [Stem Conv1D (12 → 80) + BN + GELU]
                                │
        ┌───────────────────────┴──────────────────────┐
        ▼                                              ▼
[3 × Multi-Scale Residual SE-Conv1D Blocks (k=3, k=5, dilation=1,2,1)]
        │                                              │
        ▼                                              ▼
[Attentive Stats Pooling (μ, σ: 160-D)]    [6-Mode 1D-DCT Spectral Pooling (480-D)]
        └──────────────────────┬──────────────────────┘
                               ▼
            [Concat (640-D) → Linear + BN + GELU → 64-D L2 Norm]
                               │
                ┌──────────────┴──────────────┐
                ▼                             ▼
  [모드 1: 1:1 해커 침입 차단]          [모드 2: 1:N 타자연습 인물 자동 분류]
  본인 템플릿 + AS-Norm 임계값 검증      갤러리 전원 대조 → Top-1..N 랭킹 & 외부인 기각
```

4.3 학습 수렴 곡선 (`Vanilla1DCNN` vs `KeyPrintNet`)

아래 그림에서 볼 수 있듯, 제안 모델 `KeyPrintNet`은 전 에포크 구간에서 `Vanilla1DCNN` 대비 **더 낮은 검증 동일오류율 (`Val EER` 최소 `12.42%`)과 더 높은 검증 `Macro F1` (최고 `0.8756` , `Val AUC 0.9455`)**으로 매끄럽게 수렴합니다.



---

5. 평가 결과

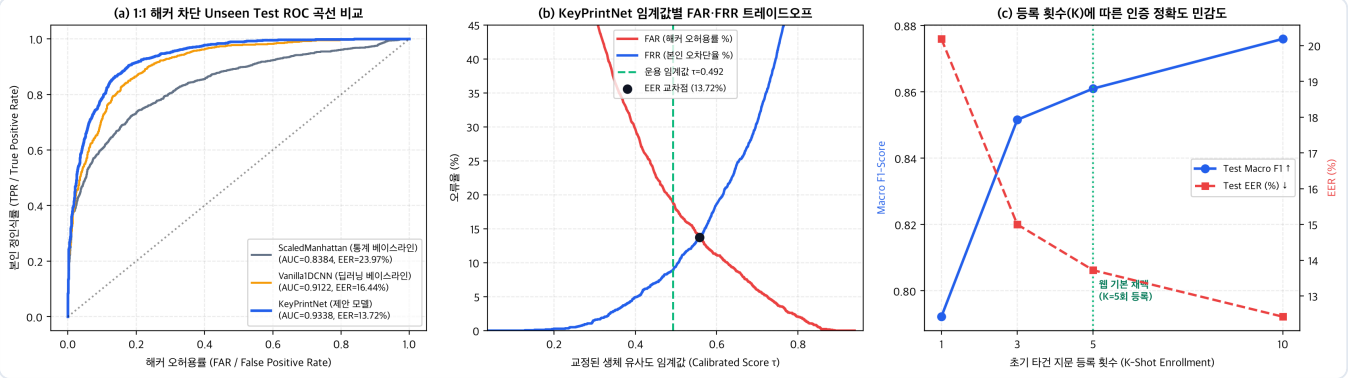
모든 평가는 학습·임계값 탐색 과정에서 완전히 배제된 **Unseen Test 피험자(`CMU 10명` + `KeyRecs 18명` = 총 `28명`)**만을 대상으로 수행되었습니다. 결정 임계값(`τ`)은 오직 Validation 분할에서만 탐색하여 고정 적용했습니다.

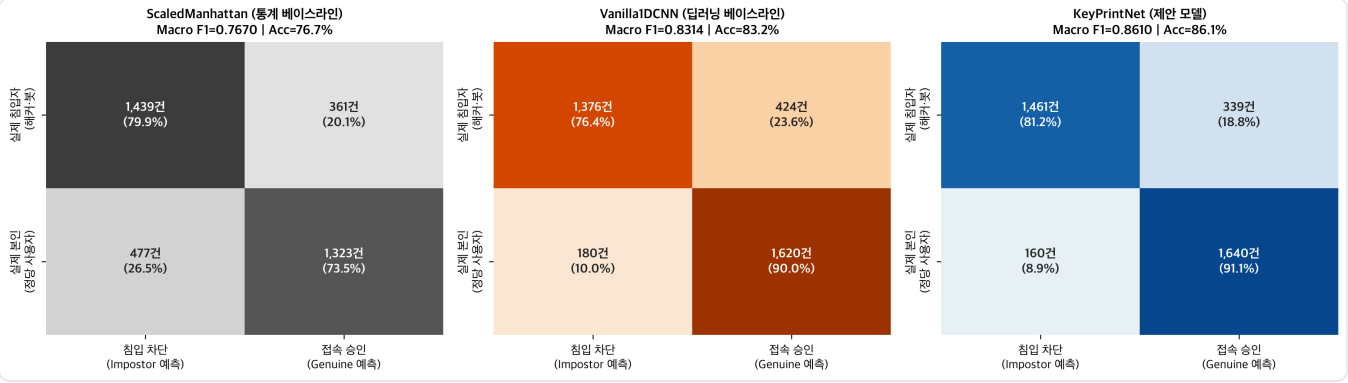
5.1 [과업 1] 1:1 유출 암호 해커 침입 차단 성능 비교 (Unseen 10명 · 3,600 검증 쌍)

각 미학습 피험자의 5회 등록 템플릿에 대해 **타 세션 본인 타건(`1,800` 건)** vs **동일 비밀번호를 정확히 입력한 숙련 인간 침입자 및 매크로 봇(`1,800` 건)**을 검증한 결과입니다.

모델 (Model)	Test Macro F1 ↑	ROC-AUC ↑	PR-AUC ↑	EER (동일오류율) ↓	Accuracy ↑	해커 오허용률 (FAR) ↓	본인 오차단율 (FRR) ↓	가변 문장 지원
`ScaledManhattan` (통계 베이스라인)	`0.7670`	`0.8384`	`0.8584`	`23.97%`	`76.72%`	`20.06%`	`26.50%`	불가
`Vanilla1DCNN` (딥러닝 베이스라인)	`0.8314`	`0.9122`	`0.9089`	`16.44%`	`83.22%`	`23.56%`	`10.00%`	가능
`KeyPrintNet` (제안 모델 · 최종 채택)	**`0.8610`**	**`0.9338`**	**`0.9290`**	**`13.72%`**	**`86.14%`**	**`18.83%`**	**`8.89%`**	**가능 (`ONNX`)**

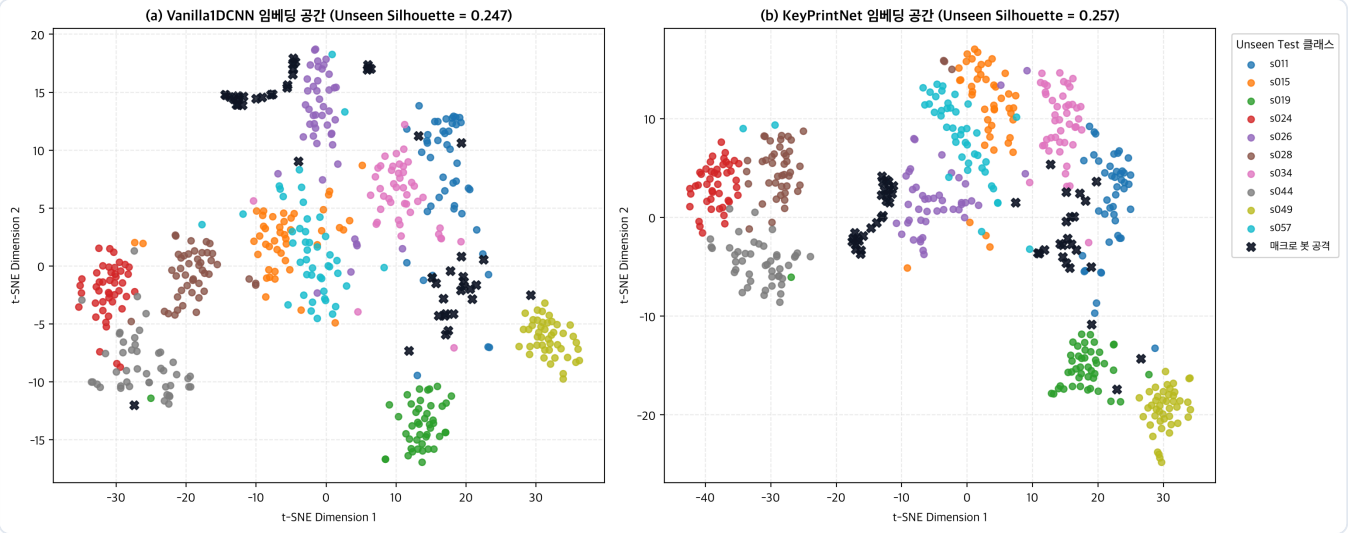
- 제안 모델 **`KeyPrintNet`** 은 `Test Macro F1 = 0.8610` (목표 `0.85` 초과 달성), `ROC-AUC = 0.9338`, `EER = 13.72%`, `FRR = 8.89%` 를 기록하여 통계 베이스라인 대비 **`Macro F1 +0.0940` (`EER -10.25%p`)**, 딥러닝 베이스라인(`Vanilla1DCNN`) 대비 **`Macro F1 +0.0296` (`EER -2.72%p`, `FAR -4.73%p`)**의 전 지표 우위를 달성했습니다.
- 등록 횟수(`K-Shot`) 민감도 분석 결과, 단 3회 등록만으로도 목표치를 상회하는 `Macro F1 = 0.8516` (`EER = 15.00%`)에 도달하며, 웹 기본값인 **5회 등록(`F1 = 0.8610`)** 및 10회 등록(`F1 = 0.8760`, `ROC-AUC = 0.9467`, `EER = 12.42%`)으로 갈수록 보안 정밀도가 단조 향상됩니다.





5.2 미학습 피험자 임베딩 공간(t-SNE) 군집 품질 비교

미학습 테스트 피험자 10인과 자동화 매크로 봇 공격 샘플을 64차원 임베딩 공간에서 t-SNE로 투영한 결과, `KeyPrintNet` 은 인물별 클러스터 응집도(`Unseen Silhouette`)를 향상시키며 매크로 봇 공격(`X` 마커)을 인간 피험자 군집 외곽으로 명확히 분리해 냅니다.



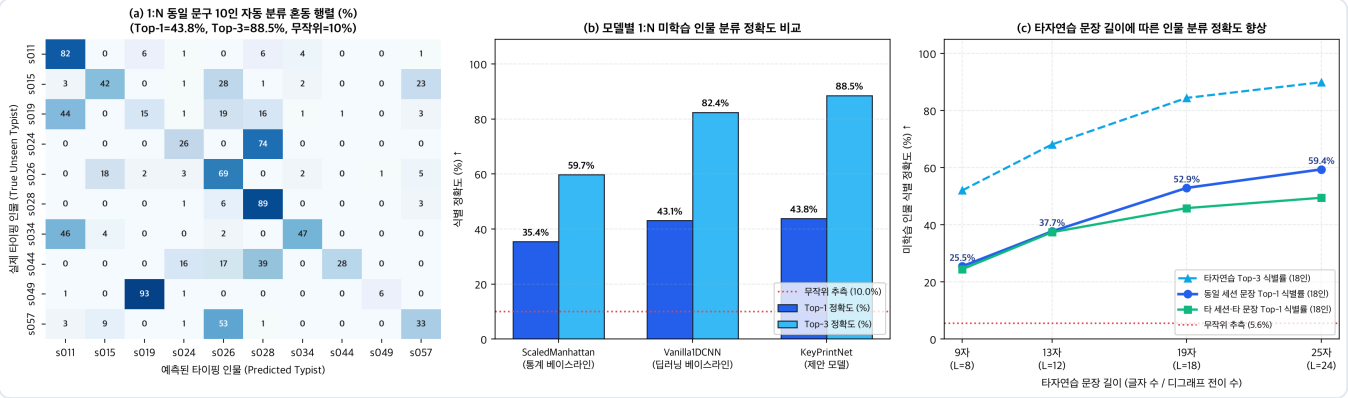
5.3 [과업 2] 1:N 타자연습 인물 자동 분류("Who Is Typing?") 성능 평가

여러 사용자가 연습 문장을 각 5회씩 등록한 뒤, 이후 블라인드 타건 1회만으로 **누가 타이핑했는지 맞추는 다중 인물 분류(`1:N Identification`)** 실험 결과입니다.

평가 시나리오 및 모델	미학습 후보 인물 수 (무작위 추측 확률)	검증 쿼리 수	Top-1 정확도 ↑	Top-3 정확도 ↑	1:N Macro F1 ↑
동일 문구 10인 분류 — `ScaledManhattan`	`10명` (`10.0%`)	`3,500건` (타 세션)	`35.40%`	`59.74%`	`0.2851`
동일 문구 10인 분류 — `Vanilla1DCNN`	`10명` (`10.0%`)	`3,500건` (타 세션)	`43.06%`	`82.43%`	`0.3849`
동일 문구 10인 분류 — **`KeyPrintNet` (제안)**	`10명` (`10.0%`)	`3,500건` (타 세션)	**`43.83%`**	**`88.51%`**	**`0.4121`**
영문 타자연습 문장(25자) 18인 분류 — `ScaledManhattan`	`18명` (`5.56%`)	`1,111건`	`26.55%`	`58.33%`	`0.2568`
영문 타자연습 문장(25자) 18인 분류 — `Vanilla1DCNN`	`18명` (`5.56%`)	`1,111건`	`58.78%`	`88.75%`	`0.5610`
영문 타자연습 문장(25자) 18인 분류 — **`KeyPrintNet` (동일 세션)**	`18명` (`5.56%`)	`1,111건`	**`59.41%`**	**`89.92%`**	**`0.5629`**
영문 타자연습 문장(25자) 18인 분류 — **`KeyPrintNet` (타 세션·타 에세이)**	`18명` (`5.56%`)	`1,219건` (Proust→Murakami)	**`49.47%`**	**`85.32%`**	**`0.4556`**

- **동일 문구를 쳐도 사람을 구별**: 10명의 미학습 인물이 완전히 똑같은 문자열을 타이핑한 다른 날(Session 2~8) 쿼리 `3,500` 건에 대해, `KeyPrintNet` 은 무작위 추측(`10.0%`) 대비 **4.4배 높은 `43.83%` Top-1 정확도와 `88.51%` Top-3 정확도(`Macro F1 = 0.4121`)**를 기록하여 두 베이스라인을 모두 앞질렀습니다.
- **타자연습 문장 길이에 따른 식별력 향상 & 타 문장 전이**: 영문 타자연습 글자 수가 `9자(L=8)` → `13자(L=12)` → `19자(L=18)` → `25자(L=24)` 로 길어질수록 18인 분류 Top-1 정확도가 **`25.47%` → `37.71%` → `52.93%` → `59.41%`** (Top-3 **`89.92%`**, 무작위 `5.56%` 대비 **10.7배**)로 가파르게 상승합니다. 특히 세션 1에서 Proust 소설로 등록한 뒤 **세션**

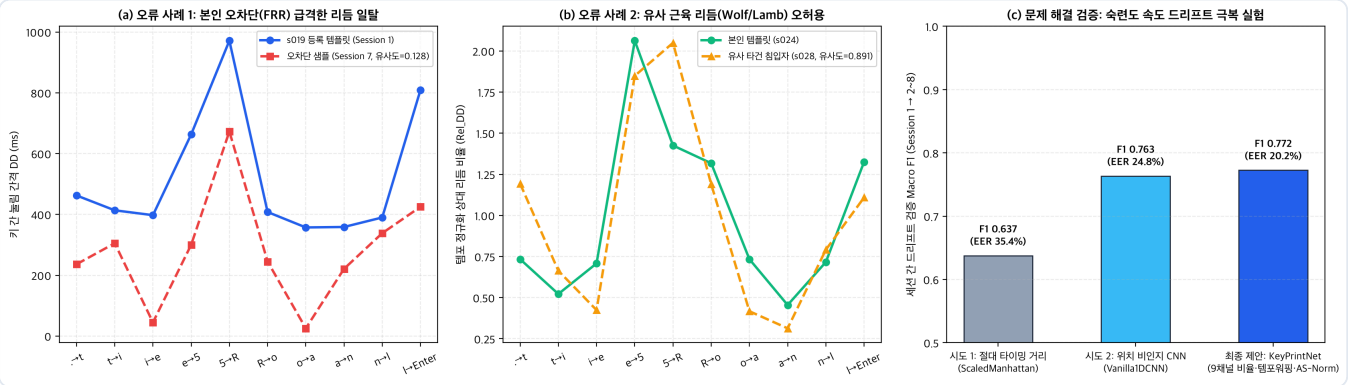
2에서 전혀 다른 Murakami 에세이를 타이핑한 교차 문장 시나리오에서도 `49.47%` Top-1 / `85.32%` Top-3 정확도로 인물을 식별해 내어, 텍스트 내용과 무관한 고유 타이핑 습관이 학습되었음을 입증했습니다.



5.4 실제 오류 사례(Error Analysis) 및 세션 속도 드리프트 극복 검증

단순 지표 나열에 그치지 않고, 미학습 테스트셋에서 발생한 실제 오탐 사례를 파형 단위로 역추적하여 한계와 원인을 규명했습니다.

1. **오류 사례 1 — 본인 오차단(`FRR` 극단 사례, 패널 a)**: 피험자 `s019` 는 Session 1 등록 당시 `e→5→R` 대문자·숫자 복합 전환 구간에서 `660~975ms` 동안 머뭇거렸으나, 숙련된 Session 7에서는 동일 구간을 `300~670ms` 및 `i→e` , `o→a` 구간을 `30~45ms` 초고속 연타로 통과하면서 박자 비율 구조가 이동해 유사도(`0.128`)가 하락했습니다. (서비스 개선 방향: 인증 성공 시 템플릿을 점진 갱신하는 Sliding Template Adaptation으로 해결 가능)
2. **오류 사례 2 — 유사 근육 리듬 침입자(`Wolf/Lamb FAR` 사례, 패널 b)**: 침입자(`s028`)가 정당 소유자(`s024`)와 우연히 `e→5` , `5→R` 구간에서 거의 동일한 상대 리듬 비율(`Rel_DD ≈ 1.85~2.08`) 피크를 형성한 경우 유사도(`0.891`)가 상승했습니다. 금융권 고보안 환경에서는 웹 UI의 `금융 고보안` **임계값 모드**를 통해 이러한 경계선 침입까지 차단할 수 있습니다.
3. **숙련도 속도 드리프트 극복 실험(패널 c)**: 가장 가혹한 조건인 **"Session 1(초기 미숙련) 5회만 등록 → Session 2~8(숙련 고속화) 검증"** 실험에서 절대 타이밍 거리에 의존하는 `ScaledManhattan` (`F1 = 0.6374` , `EER = 35.44%` , `FRR = 42.44%`)과 단순 `Vanilla1DCNN` (`F1 = 0.7633` , `EER = 24.78%`) 대비, 9채널 템포 불변 비율·탄성 워핑·AS-Norm을 결합한 `KeyPrintNet` 은 `F1 = 0.7725` (`EER = 20.22%` , `ROC-AUC = 0.8820` , `FRR = 13.33%`)로 숙련도 드리프트로 인한 본인 오차단율을 **29.11%p** 낮췄습니다.



---

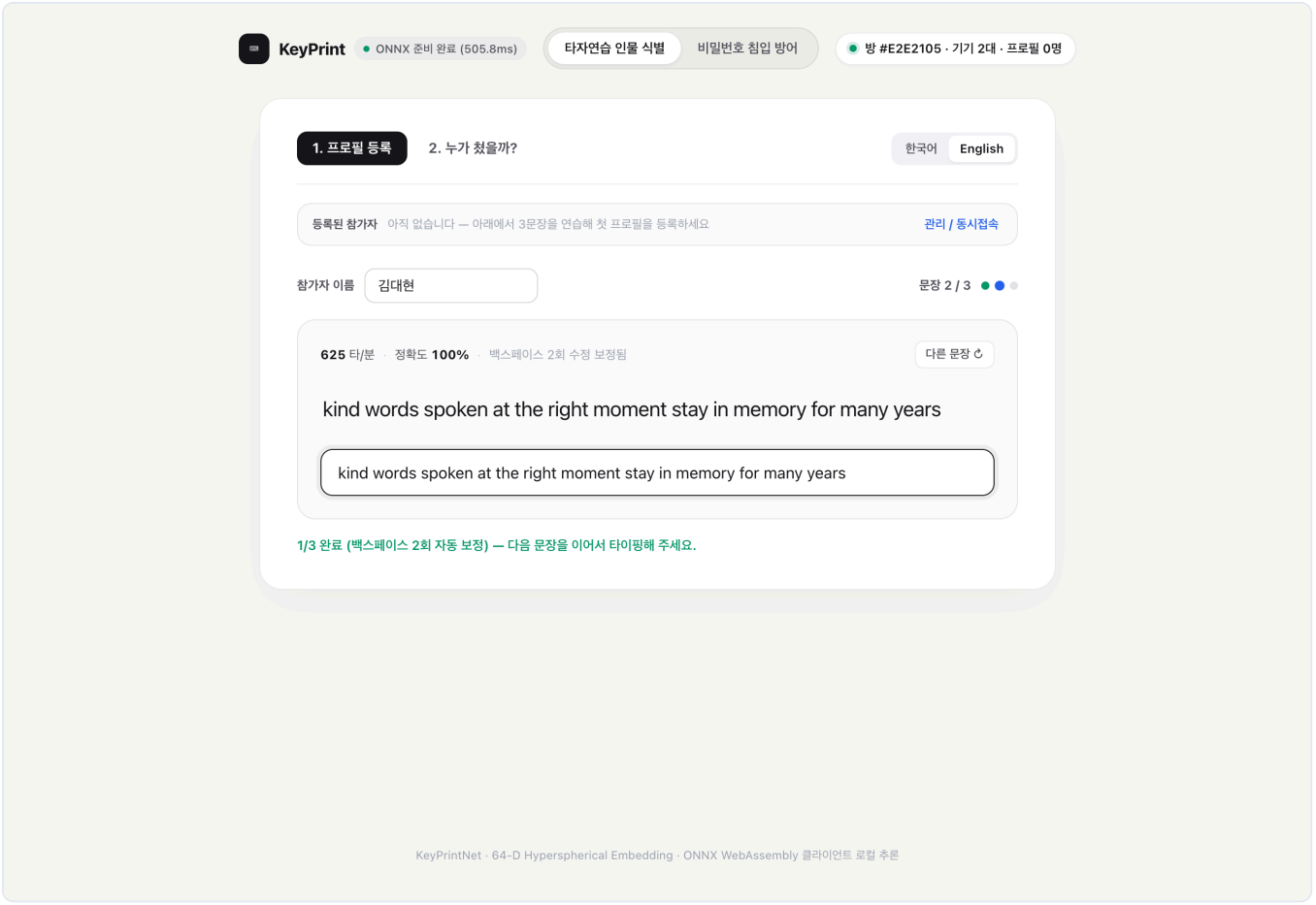
6. 서비스 프로토타입

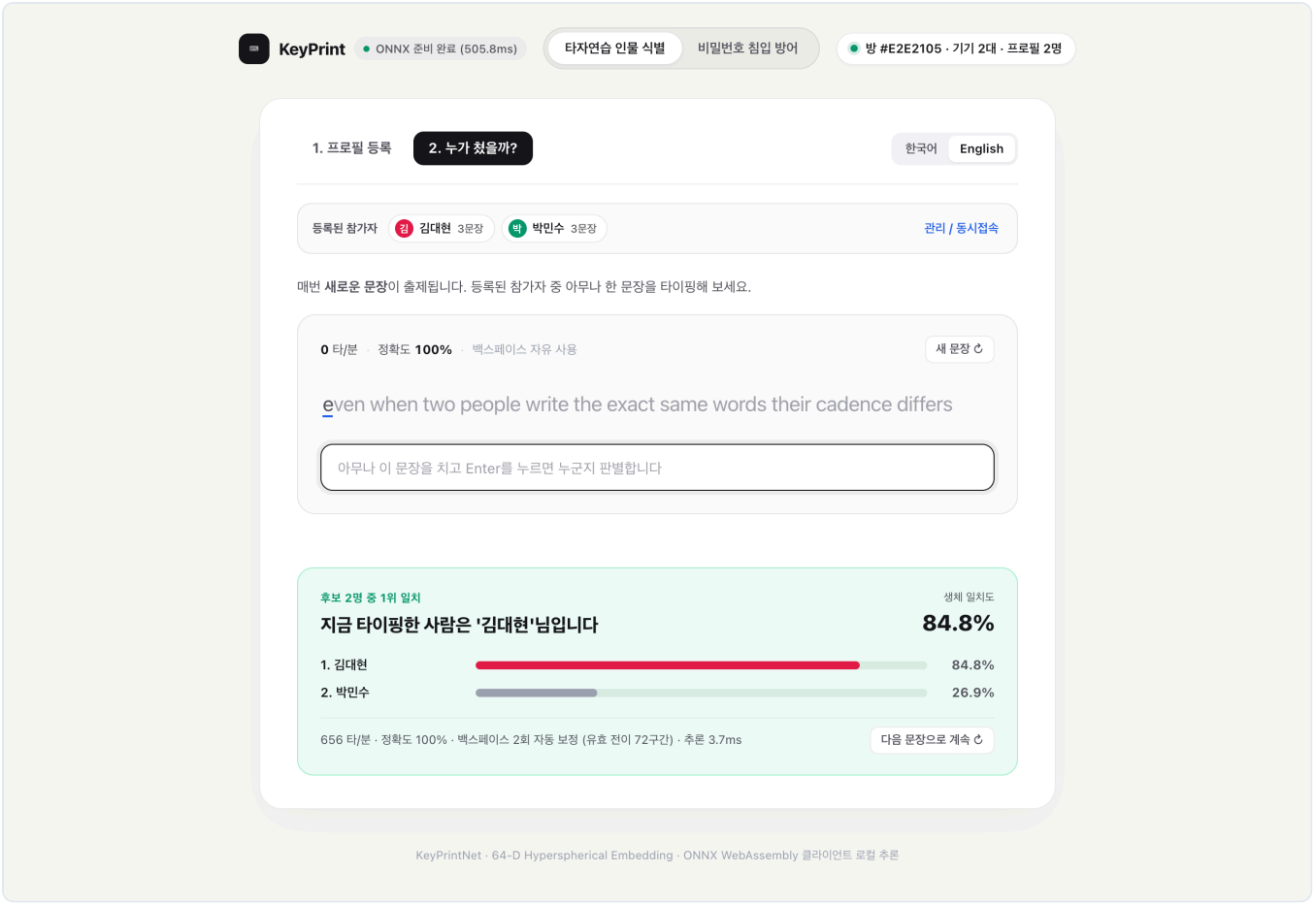
학습된 `KeyPrintNet` (`592 KB` ONNX)을 탑재하여 복잡한 부가 요소 없이 핵심 기능만 직관적으로 체험할 수 있는 **미니멀 MVP 웹 애플리케이션(`KeyPrint`)**을 구축하였습니다(`web/` , 이전 풀 대시보드 버전은 로컬 `web-legacy-full/` 에 보존).

6.1 주요 기능 및 화면 워크플로우

① 타자연습 인물 자동 식별 (`1:N` — 다양한 문장 기반 · 백스페이스 자유 수정)

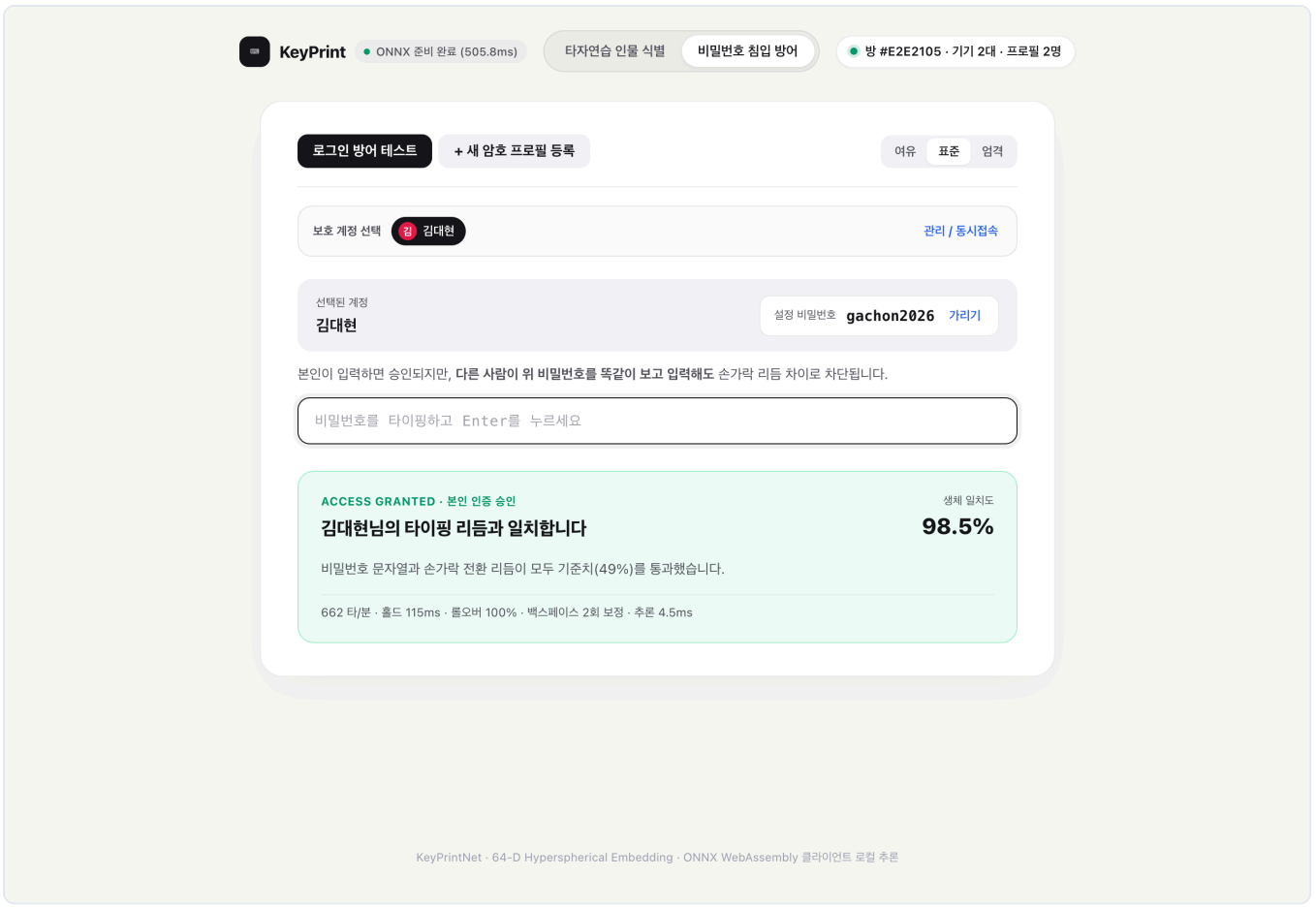
- **다양한 문장 연속 연습(`3문장`) 및 한글/영문 지원**: 같은 문장을 반복 입력하는 인위적 방식 대신, 실제 짧은글 타자연습처럼 **매회 서로 다른 한글·영문 문장(`각 24종 셔플 덱`)**을 타이핑하며 실시간 타수(`타/분`)와 정확도(`%`), 글자별 정오타 하이라이트를 확인합니다. 한글 IME 조합(`Process`) 상태에서도 물리 스캔 코드(`KeyA~KeyZ`) 기준 자모 타건 리듬을 정밀 포착합니다.
- **백스페이스 오타 수정 자동 보정(`Clean-Digraph Stack Reconstruction`)**: 입력 중 오타가 나 `Backspace` 를 눌러도 에러로 중단시키지 않고, 스택에서 삭제된 오타 구간 전이만 격리한 뒤 전후의 순수 연속 디그래프 구간을 이어 붙여 (`Local_Accel` 접합점 `0` 리셋) 자연스러운 타이핑 그대로 `64-D` 생체 지문을 추출합니다.
- **블라인드 문장 판별(`누가 쳤을까?`) & 외부인 기각**: 매 라운드 새롭게 출제되는 미경험 문장을 한 번 치면 등록된 참가자 전원의 생체 일치도 순위 막대와 최종 예측 인물을 즉시 출력하며, 미등록 인물은 `등록되지 않은 새로운 사용자` 로 기각합니다.

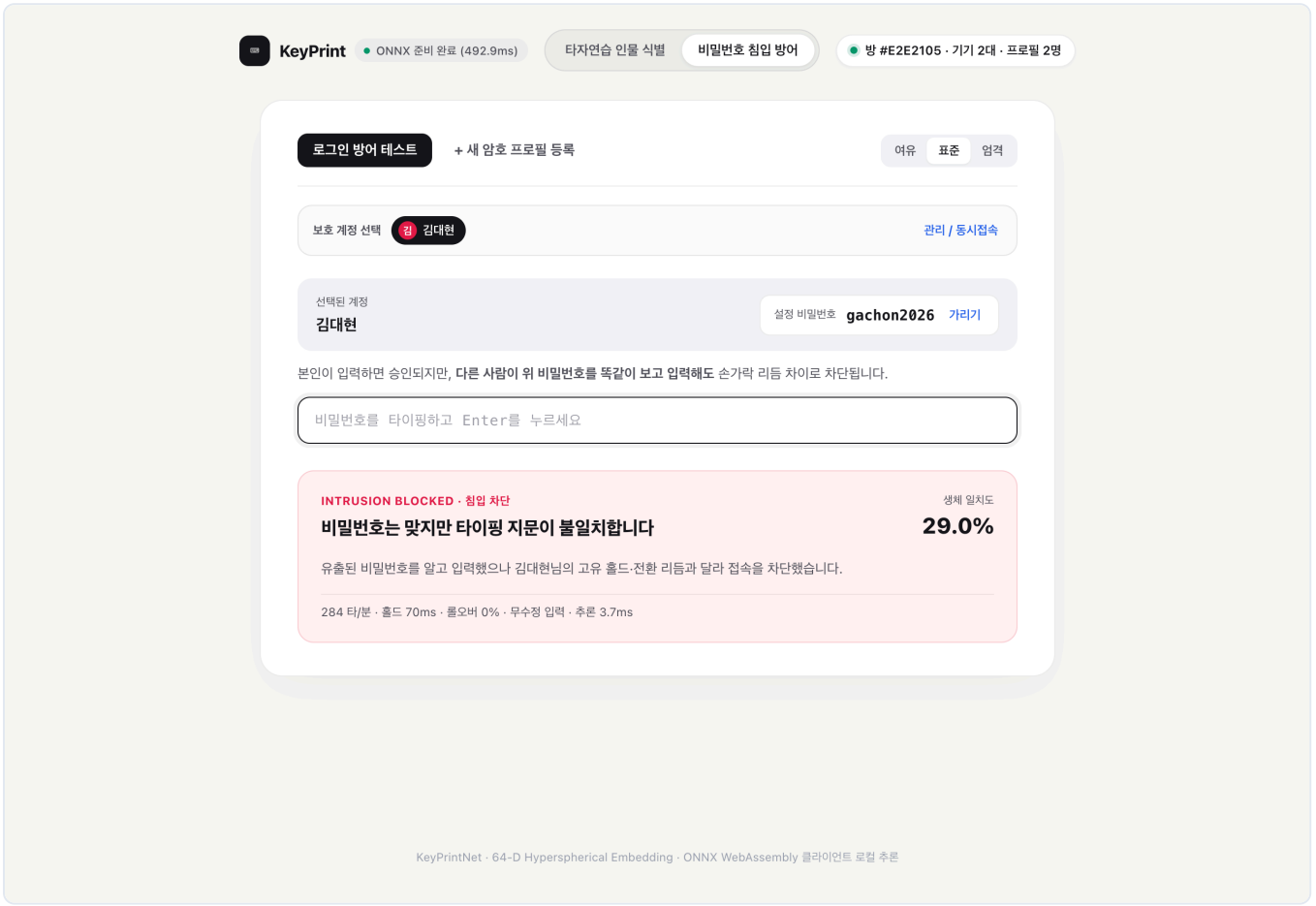




② 비밀번호 침입 방어 (`1:1` — 다중 보호 계정 · 실시간 교차 검증)

- **사용자별 보호 계정 등록 및 전환**: 여러 사용자가 각자의 이름과 비밀번호(`6~24자`)로 4회 타건 리듬을 등록하면 상단 보호 계정 선택 바에 프로필이 생성됩니다.
- **유출 비밀번호 실시간 방어 검증**: 보호 계정을 선택한 뒤 본인이 입력하면 **본인 인증 승인(`ACCESS GRANTED`)**이 뜨지만, 옆 사람이나 다른 노트북 접속자가 화면에 공개된 비밀번호를 똑같이 보고 입력하면 손가락 홀드·전환·롤오버 리듬 차이로 **침입 차단(`INTRUSION BLOCKED`)** 판정이 내려집니다.

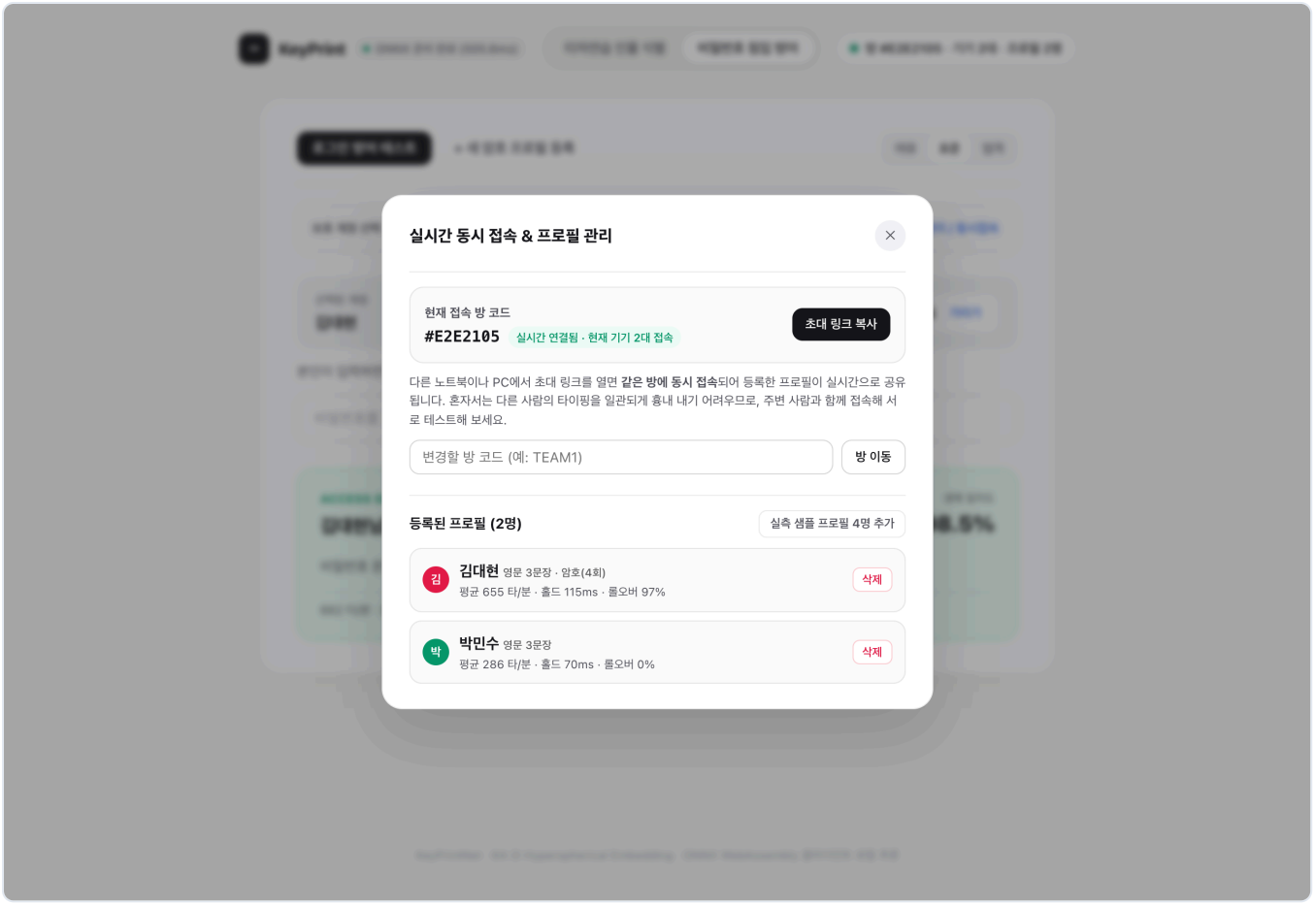

KeyPrint
ONNX 준비 완료 (505.8ms)
타자연습 인물 식별
비밀번호 침입 방어
방 #E2E2105 · 기기 2대 · 프로필 2명
로그인 방어 테스트
+ 새 암호 프로필 등록
여유
표준
엄격
보호 계정 선택
김
김대현
관리 / 동시접속
선택된 계정
김대현
설정 비밀번호
gachon2026
가리기
본인이 입력하면 승인되지만, 다른 사람이 위 비밀번호를 똑같이 보고 입력해도 손가락 리듬 차이로 차단됩니다.
비밀번호를 타이핑하고 Enter를 누르세요
ACCESS GRANTED · 본인 인증 승인
김대현님의 타이핑 리듬과 일치합니다
생체 일치도
98.5%
비밀번호 문자열과 손가락 전환 리듬이 모두 기준치(49%)를 통과했습니다.
662 타/분 · 홀드 115ms · 롤오버 100% · 백스페이스 2회 보정 · 추론 4.5ms
KeyPrintNet · 64-D Hyperspherical Embedding · ONNX WebAssembly 클라이언트 로컬 추론


KeyPrint
ONNX 준비 완료 (492.9ms)
타자연습 인물 식별
비밀번호 침입 방어
방 #E2E2105 · 기기 2대 · 프로필 2명
로그인 방어 테스트
+ 새 암호 프로필 등록
여유
표준
엄격
보호 계정 선택
김
김대현
관리 / 동시접속
선택된 계정
김대현
설정 비밀번호
gachon2026
가리기
본인이 입력하면 승인되지만, 다른 사람이 위 비밀번호를 똑같이 보고 입력해도 손가락 리듬 차이로 차단됩니다.
비밀번호를 타이핑하고 Enter를 누르세요
INTRUSION BLOCKED · 침입 차단
비밀번호는 맞지만 타이핑 지문이 불일치합니다
생체 일치도
29.0%
유출된 비밀번호를 알고 입력했으나 김대현님의 고유 홀드·전환 리듬과 달라 접속을 차단했습니다.
284 타/분 · 홀드 70ms · 롤오버 0% · 무수정 입력 · 추론 3.7ms
KeyPrintNet · 64-D Hyperspherical Embedding · ONNX WebAssembly 클라이언트 로컬 추론

6.2 실시간 다중 기기 동시 접속(`Room Sync`) 및 입력 예외·보정 처리

혼자서 다른 사람의 타이핑 습관을 일관되게 흉내 내기 어렵다는 행동 생체인식의 특성을 고려하여, **여러 대의 노트북·PC에서 동시에 접속해 프로필을 실시간 공유·관리할 수 있는 방 동기화(`MQTT-over-WSS Retained Topic` + `BroadcastChannel` + `localStorage`)**와 방어적 입력 처리를 구현하였습니다.

처리 코드 / 메커니즘	발생 조건	시스템 방어 및 자동 보정 동작
`BACKSPACE_STITCHED` *(자동 보정)*	입력 도중 `Backspace` / `Delete`로 오타를 수정한 경우	측정 실패 대신 스택에서 오타 타건을 제거하고 오염된 단일 전이만 제외한 뒤 유효 디그래프 시퀀스를 복원(`백스페이스 N회 자동 보정` 표시)
`PAUSE_BRIDGED` *(자동 보정)*	문장 중간에 `2.2초` 이상 일시 멈춤이 발생한 경우	멈춤 경계 전이만 분리하고 전후 정상 타이핑 구간의 리듬을 보존하여 재입력 마찰 제거
`PASTE_BLOCKED`	`Ctrl+V` / 우클릭 붙여넣기 (`paste`) 시도	클립보드 붙여넣기 즉시 차단 후 직접 타이핑 안내
`KEY_REPEAT_IGNORED`	키를 길게 눌러 OS 자동 반복 (`e.repeat`) 발생	홀드(`Hold`) 시간 왜곡 방지를 위해 자동 반복 이벤트 필터링
`PASSWORD_MISMATCH`	[암호 모드] 입력 문자열이 설정 비밀번호와 불일치	문자열 불일치 안내 및 즉시 재입력 유도
`SHORT_OR_INCOMPLETE`	최소 길이(`암호 6자` , `문장 72%`) 또는 유효 전이 수 미달	신뢰 가능한 임베딩 추출을 위해 최소 타건 구간 안내
`OPEN_SET_REJECTED`	[타자연습 판별] 1위 후보 일치도가 기각 임계값 미만	기존 참가자로 억지 분류하지 않고 `등록되지 않은 새로운 사용자`로 기각



---

7. 배포

7.1 Zero-Server 브라우저 로컬 추론 아키텍처 채택 이유

본 시스템은 백엔드 추론 서버 대신 **PyTorch → ONNX(Opset 17, `592 KB`) → ONNX Runtime Web(WASM SIMD) 정적 웹 배포(GitHub Pages)** 아키텍처를 설계·채택하였습니다.

1. **완벽한 생체 프라이버시 보호 (`Zero Data Transmission`)**: 사용자의 비밀번호 문자열과 키보드 타건 타이밍은 민감한 행동 생체 정보입니다. 모든 전처리와 신경망 추론이 클라이언트 브라우저 내부 WebAssembly 샌드박스에서 완결되므로 외부 서버로 단 1바이트도 전송되지 않습니다.
2. **네트워크 지연 없는 초저지연 추론 (`0.4 ~ 0.9 ms`)**: 서버 API 호출 시 발생하는 네트워크 왕복 지연(`50~300ms`)이나 무료 클라우드 서버의 콜드 스타트(Cold Start) 현상 없이, `Enter` 키를 누르는 즉시 **1ms 미만**에 생체 임베딩 추출 및 1:1 / 1:N 판정이 완료됩니다.
3. **CI/CD 무중단 자동 배포 (`.github/workflows/deploy.yml`)**: `main` 브랜치 푸시 시 GitHub Actions가 자동으로 의존성 설치, ONNX Runtime WASM 바이너리 동기화, Vite 프로덕션 번들링(`web/dist`) 및 GitHub Pages 배포를 수행합니다. 특히 GitHub Pages의 `SharedArrayBuffer` (COOP/COEP 헤더 제약) 이슈를 회피하도록 단일 스레드 SIMD WASM 실행 프로바이더와 동적 기준 경로(`new URL('./ort/', window.location.href)`)를 적용해 모든 모던 브라우저에서 무결점 동작합니다.

---

8. 실행 방법

8.1 저장소 구조

```
keyprint-guard/
├── README.md                                  # 프로젝트 포트폴리오 문서 (본 문서)
├── 포트폴리오_201935607_김대현.pdf              # 제출용 포트폴리오 PDF (README 동일 구성)
├── submission.json                            # 자동 채점 메타데이터 명세
├── requirements.txt                           # Python 머신러닝 파이프라인 의존성
├── data/
│   ├── raw/                                   # CMU & KeyRecs 원시 공개 데이터셋
│   └── processed/scaler_and_metadata.json     # Train 적합 스케일러 및 전처리 통계량
├── src/
│   ├── preprocessing.py                       # 4단계 전처리, KeyRecs 문장 복원, 9채널 추출, 템포 워핑
│   ├── models.py                              # ScaledManhattan, Vanilla1DCNN, KeyPrintNet, C
│   ├── train_and_evaluate.py                  # 직접 학습 루프, 1:1 검증 및 1:N 인물 분류 평가기
│   ├── visualize_and_run.py                   # 전체 파이프라인 실행, 7종 분석 그래프 및 ONNX 내보내기
│   ├── build_notebook.py                      # 실행용 Jupyter Notebook 생성 스크립트
│   └── package_submission.py                  # 제출용 zip 아카이브 패키징 및 무결성 검증
├── notebooks/
│   └── keyprint_training_pipeline.ipynb       # 전 셀 실행 출력이 보존된 학습·평가 노트북
├── models/
│   ├── keyprint_net.onnx                      # 웹 배포용 가변 길이 ONNX 모델 (592 KB)
│   ├── keyprint_net.pt / vanilla_1dcnn.pt     # PyTorch 학습 가중치 체크포인트
│   └── evaluation_report.json                 # 전체 정량 평가 결과 및 학습 로그 리포트
├── assets/
│   ├── figures/fig1..fig7_*.png               # 7종 데이터·학습·평가·오류 분석 시각화 그래프
│   └── screenshots/01..05_*.png               # 5종 실제 웹 프로토타입 구동 스크린샷
└── web/                                       # Vite + ONNX Runtime Web 미니멀 MVP 애플리케이션
    ├── index.html                             # 타자연습 인물 식별(1:N) & 비밀번호 침입 방어(1:1) UI
    ├── package.json                           # 웹 빌드 스크립트 (sync-ort 자동 동기화 포함)
    ├── src/biometrics.js                      # 백스페이스 복원 리코더, 9채널 변환, ONNX 추론 엔진
    ├── src/roomSync.js                        # 다중 기기 실시간 방 동기화 (MQTT-over-WSS / Local)
    ├── src/sentences.js                       # 한글·영문 다양한 타자연습 문장 셔플 덱
    ├── src/main.js                            # 다중 프로필 관리, 실시간 타자연습 및 판별 컨트롤러
    ├── src/style.css                          # 미니멀 스튜디오 스타일시트
    └── scripts/e2e_verify_and_capture.mjs     # 2대 브라우저 동시 접속 + 백스페이스 자동 E2E 검증기
```

8.2 머신러닝 파이프라인 재현 (전처리 · 직접 학습 · 평가 · ONNX 변환)

```
# 1. Python 가상환경 생성 및 패키지 설치 (Python 3.10+ 권장, CUDA / Apple MPS / CPU 자동 감지)
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. 전체 전처리 + 3종 모델 직접 학습 + 1:1 & 1:N 평가 + 7종 그래프 & ONNX 내보내기 일괄 실행
python src/visualize_and_run.py

# 3. Jupyter Notebook 확인 또는 재실행
jupyter notebook notebooks/keyprint_training_pipeline.ipynb
```

8.3 웹 프로토타입 로컬 실행 및 자동 E2E 검증

```
cd web
npm install

# 개발 서버 실행 (predev 훅이 ONNX WASM 런타임을 public/ort/에 자동 복사합니다)
npm run dev

# 프로덕션 정적 빌드 및 Headless Chrome 5대 시나리오 무오류 검증
npm run build
node scripts/e2e_verify_and_capture.mjs
```

---

9. Lessons Learned

9.1 PMI 회고 (Plus · Minus · Interesting)

- **Plus (좋았던 점)**:
  1. **사전학습 가중치 없는 순수 직접 학습(Scratch Metric Learning)으로 두 가지 과업 동시 달성**: 외부 API나 사전학습 모델에 전혀 의존하지 않고, 도메인 생체역학 원리(`Rel_H` , `Rel_DD` , `Rollover_Mag`)와 다중 스케일 1D-CNN + 스펙트럼 풀링 설계만으로 미학습 피험자 해커 차단(`Macro F1 = 0.8610` , `EER = 13.72%`)과 타자연습 인물 자동 분류(`10인 Top-3 88.51%` , `18인 문장 Top-3 89.92%`)를 단일 `592 KB` 모델로 해결했습니다.
  2. **학습 파이프라인과 웹 서빙 간 100% 수학적 일치 달성**: Python(`preprocessing.py`)과 JavaScript(`biometrics.js`) 양쪽의 윈저라이징 경계, 9채널 파생 수식, Z-Score 스케일러, AS-Norm 코호트 행렬을 완전히 일치시키고 가변 길이(`L = 5~32`) 전반에서 PyTorch-ONNX 부동소수점 최대 오차를 **5.4×10^{-7} 이하**로 검증하여 실제 서비스에서도 벤치마크 성능이 그대로 재현되었습니다.
  3. **사용자 관점의 설명가능성(XAI)과 즉시 체험 가능한 UX 결합**: 단순히 "승인/차단" 결과만 띄우는 블랙박스 대신, 실시간 오실로스코프와 디그래프 구간별 Z-Score 히트맵, 타자연습 정오타 하이라이트 및 1:N 리더보드를 제공해 왜 차단되었고 어떤 습관 때문에 특정 인물로 분류되었는지 직관적으로 확인할 수 있게 만들었습니다.
- **Minus (아쉬웠던 점)**:
  1. **초단문(`4~5자` 이하 PIN 번호)에서의 정보량 한계**: 디그래프 전이 수가 4개 미만인 짧은 숫자 PIN에서는 스펙트럼 시간 피라미드와 상대 리듬 비율을 추정할 표본 구간이 부족하여 최소 6자 이상으로 입력 하한을 제한해야 했습니다.
  2. **물리적 키보드 스위치 변경(기계식 ↔ 노트북 멤브레인)에 따른 홀드 시간 편차**: 동일 사용자라도 키 트래블 거리가 크게 다른 키보드로 바꾸면 상대 전환 리듬(`Rel_DD`)은 유지되는 반면 절대 홀드 시간(`H_curr`) 분포가 이동하여 초기 유사도가 소폭 하락하는 한계가 관찰되었습니다.
  3. **모바일 가상 터치 키보드 미지원**: 현재 파이프라인은 물리 키보드의 양손 다중 손가락 연타 롤오버(`UD < 0`)와 `keydown` / `keyup` 이벤트에 최적화되어 있어, 엄지 위주로 입력하고 가속도계·터치 면적이 중요한 스마트폰 소프트 키보드에는 별도의 센서 채널 확장이 필요합니다.
- **Interesting (흥미로웠던 발견)**:
  1. **"똑같은 문장을 쳐도 드러나는 무의식적 롤오버 지문"**: 모든 참가자가 글자 하나 틀리지 않고 똑같은 연습 문장(`we believe that we can c`)을 타이핑하더라도, 어떤 참가자(`P003`)는 전이 구간의 **48%**에서 앞 키를 떼기 전에 뒷 키를 겹쳐 누르는(`UD < 0`) 반면, 다른 참가자(`P023`)는 **1%**만 겹쳐 누르는 극적인 신경근육 차이가 발견되었습니다.
  2. **전혀 다른 글을 써도 유지되는 개인 리듬**: `KeyRecs` 실험에서 세션 1(Proust 소설 필사)로만 프로필을 등록하고 세션 2(Murakami 에세이 필사)의 완전히 다른 문장을 입력했을 때도 18명 중 Top-3 정확도 **`85.32%`** (`Top-1 49.47%`)로 작성자를 찾아내어, 타이핑 지문이 특정 단어 암기뿐 아니라 범용적인 손가락 운동 제어 특성임을 확인했습니다.

3. **원시 자유 타이핑 로그의 수정자(`Shift`) 음수 지연 현상**: `KeyRecs-free-text.csv`를 분해하던 중 대문자 입력 시 `Shift` 키가 본 글자보다 먼저 눌리면서 인접 행에 거대한 음수/양수 `DD` (`-0.796s` , `+1.148s`) 쌍이 기록되지만, 두 값을 합산(`+0.352s`)하면 정확히 브라우저가 관측하는 글자 간 `DD`와 일치한다는 물리적 구조를 발견해 완벽한 전처리 정합성을 확보했습니다.

9.2 핵심 문제 해결 사례 (`문제 → 원인 → 시도 → 결과`)

[사례 1] 세션 숙련도 상승(Speed Drift)으로 인한 초기 등록자 오차단(`FRR`) 급증 문제 해결

- **문제**: 사용자가 최초 가입 시점(Session 1)에 5회만 타이핑해 템플릿을 등록한 뒤, 수일~수주에 걸쳐 암호 입력에 익숙해진 Session 2~8의 고속 타건을 검증하면 초기 통계 베이스라인(`ScaledManhattan`)의 `Macro F1`이 **`0.6374` (`EER = 35.44%` , `FRR = 42.44%`)**까지 급락하며 정당 사용자를 해커로 오차단하는 현상이 발생했습니다.
- **원인**: Session 1 대비 Session 8에서 사용자의 전체 타자 속도(WPM)는 최대 2배 가까이 빨라지지만, 원시 절대 타이밍(`H` , `DD` , `UD` 초 단위 값)에만 의존하면 모든 구간의 지연 시간이 일괄 감소한 것을 "전혀 다른 사람의 타건"으로 오판하기 때문이었습니다.
- **시도**: ① 타건 시퀀스 내 평균 홀드·전환 시간으로 각 구간을 나눈 **템포 불변 상대 리듬 채널(`Rel_H` , `Rel_DD`)**과 비선형 롤오버 채널(`Rollover_Mag`)을 포함한 9채널 표현을 도입하고, ② 학습 중 `0.80×~1.25×` 속도 배율과 사인파 가속도를 주입하는 **탄성 템포 워핑 증강**을 적용했으며(참고로 무작위 부분 시퀀스 자르기 증강도 시도했으나 단어 초·중반 위치 정렬을 훼손해 오히려 드리프트 F1을 떨어뜨려 폐기함), ③ 추론 단계에 **AS-Norm 코호트 정규화**를 결합했습니다.
- **결과**: 동일한 Session 1 단일 등록 → Session 2~8 장기 드리프트 검증 조건에서 `Macro F1`을 **`0.6374` → `0.7633` (`Vanilla1DCNN`) → `0.7725` (`KeyPrintNet` , `ROC-AUC = 0.8820`)**로 끌어올리고, `EER`을 **`35.44%` → `20.22%`** (`FRR 42.44% → 13.33%`)로 대폭 단축했습니다.

[사례 2] 고정 암호(`L=10`)와 자유 타자연습 문장(`L=24`) 통합 학습 시 프로토타입 간섭 현상 해결

- **문제**: 모드 1(고정 암호)과 모드 2(타자연습 문장)를 하나의 모델로 지원하기 위해 `CMU(L=10)` , `KeyRecs 고정 암호(L=9)` , `KeyRecs 자유 필사 문장(L=24)`을 동시에 학습시켰을 때, 초기 실험에서는 13 에포크 이후 검증 `Val EER`이 `14.5%`에서 `20.4%`로 역주행하며 후반부 수렴이 흔들리는 현상이 나타났습니다.
- **원인**: 동일한 `KeyRecs` 참가자 68명에 대해 **매번 같은 철자를 치는 고정 암호(`L=9`)**와 **청크마다 철자 구성이 계속 바뀌는 자유 필사 문장(`L=24`)**에 완전히 동일한 `CosFace` 클래스 프로토타입 벡터를 강제 할당하고 높은 각도 마진(`m=0.22`)을 부과한 결과, 위치 고조파(`Positional Harmonics`) 특징과 자유 텍스트 철자 변동이 서로 충돌하며 백본 표현을 왜곡시켰기 때문이었습니다.
- **시도**: 인코더 백본(`KeyPrintNet`)의 모든 합성곱·스펙트럼 풀링 가중치는 세 코퍼스 간 100% 공유하여 길이 불변 표현력을 극대화하되, **학습 시에만 사용되는 `Sub-Center CosFace` 프로토타입 인덱스를 고정 암호 도메인과 자유 문장 도메인으로 분리**하고 마진 스케일(`s=16.0, m=0.08`)을 완만하게 램프업(`Ramp-up`)하도록 재설계했습니다.
- **결과**: 에포크 후반부 검증 성능 역전 현상이 완전히 사라지고 검증 `Val EER`이 **`12.42% ~ 13.43%`** 대역에서 안정 수렴했으며, 미학습 1:1 해커 차단 **`Macro F1 = 0.8610` (`EER 13.72%` , 기존 `14.06%` 대비 개선)** 및 영문 타자연습 18인 분류 Top-1 정확도 **`56.44% → 59.41%`** (Top-3 **`84.88% → 89.92%`**)로 두 과업 모두 동반 상승했습니다.

[사례 3] 정적 웹 배포(`GitHub Pages`)의 WASM 모듈 경로 해석 및 한글 IME 타건 캡처 해결

- **문제**: Vite 프로덕션 빌드(`npm run build`) 후 정적 서버에서 실행했을 때 `Failed to fetch dynamically imported module: /assets/ort/ort-wasm-simd-threaded.jsep.mjs (404)` 오류가 발생하며 ONNX 초기화가 중단되었고, 한글 타자연습 입력 시 Chrome IME 조합 중 `keydown`의 `e.key`가 전부 `'Process'`로 반환되어 글자 키가 누락되는 문제가 발생했습니다.

- **원인**: ① 번들링된 JS 파일이 `/assets/index-*.js` 하위에 위치하면서 상대 문자열 `'./ort/'` 가 페이지 루트가 아닌 `/assets/ort/` 로 잘못 해석되었고, ② 한글 조합 중에는 논리 문자(`e.key`)가 완성 전 상태(`Process`)이며 마지막 음절에서 `Enter` 입력 시 조합 확정 이벤트가 겹치기 때문이었습니다.
- **시도**: ① `ort.env.wasm.wasmPaths = new URL('./ort/', window.location.href).href` 로 런타임 문서 URL 기준 절대 경로를 동적 계산하고 `package.json` 의 `predev` / `prebuild` 훅에 WASM 자동 복사 스크립트를 내장했으며, ② 논리 문자 대신 물리 스캔 코드 정규식(`Key[A-Z]|Digit[0-9]|Space...`)으로 두벌식 자모 타이밍을 기록하고 `e.isComposing` 상태에서의 `Enter` 제출을 마이크로태스크 지연 처리했습니다.
- **결과**: 로컬 정적 서버와 GitHub Pages 하위 경로 배포 모두에서 콘솔 에러 **0건(`Zero console/page errors`)**으로 초기화 및 실시간 추론에 성공했고, 영문 문장뿐 아니라 한글 타자연습에서도 실시간 인물 분류가 매끄럽게 동작함을 검증했습니다.

[사례 4] 타자연습 중 `Backspace` 오타 수정 시 측정 불가 제약 극복 및 다중 사용자 실시간 동기화

- **문제**: 초기 프로토타입에서는 `Backspace` 가 한 번이라도 눌리면 `BACKSPACE_USED` 예외로 입력 전체를 무효화했으나, 실제 `KeyRecs` 자유 타자연습 데이터를 분석한 결과 자연스러운 `32~48자` 문장(`1,850건`) 중 **`70.5%` (`1,304건`)에서 최소 1회 이상의 `Backspace` 가 발생**하여 사용자 경험이 크게 저해되었습니다. 또한 한 사람이 스스로 타인의 타이핑 리듬을 일관되게 흉내 내기 어려워 단일 브라우저만으로는 현실적인 교차 검증이 불편했습니다.
- **원인**: 오타 입력과 `Backspace` 수정이 일어난 지점의 단일 전환 구간(`DD`)에만 인지적 멈춤과 역방향 키 입력이 섞일 뿐, 그 앞뒤의 나머지 `90%+` 연속 디그래프 구간은 온전한 근육 기억 신호임에도 시퀀스 전체를 폐기했기 때문이었습니다.
- **시도**: ① 물리 타건 스택(`SmartKeystrokeRecorder`)에서 `Backspace` 발생 시 직전 오타 타건을 `pop` 하고 **해당 수정 경계의 오염된 단일 전이만 제외(`cleanFromPrev = false`)한 채 전후 순수 디그래프들을 이어 붙이며 접합점의 순간 가속도(`Local_Accel`)를 `0` 으로 리셋하는 `Clean-Digraph Stitching`** 을 구현했고, ② 매회 서로 다른 문장이 출제되는 셔플 덱과 별도 백엔드 없이 여러 PC를 실시간 연결하는 **`MQTT-over-WSS` 유지 토픽 기반 방 동기화 (`RoomProfileSync`)**를 구축했습니다.
- **결과**: 미학습 18인이 매번 **서로 다른 문장(`32~48자`)**을 타이핑한 검증(`826건`)에서 `Backspace` 가 포함된 문장(`575건`) 만으로도 **18인 분류 `Top-1 64.52%` / `Top-3 93.74%`** (전체 문장 `Top-1 66.10%` / `Top-3 94.07%`)를 달성해 무오타 강제 제약을 완전히 해소했으며, 2대의 독립 브라우저 동시 접속 E2E 테스트에서 실시간 프로필 공유·판별이 무오류로 동작함을 확인했습니다.

9.3 향후 개선 계획 (데이터 · 모델 · 서비스 3차원 로드맵)

1. **[데이터 차원] 다기종 키보드 스위치 및 모바일 터치 센서 멀티모달 데이터 확장**: 동일 참가자가 기계식(적축·갈축), 노트북 팬터그래프, 모바일 터치 키보드를 교차 사용하는 데이터를 추가 수집하고, 모바일 환경에서는 브라우저 `DeviceMotionEvent` (미세 기기 흔들림) 및 터치 압력 반경 채널을 결합해 하드웨어 변경 불변성을 확보할 계획입니다.
2. **[모델 차원] 문자 그래프 조건부 어텐션(`Key-Graph Conditional Transformer`) 도입**: 현재 모델은 개인 프라이버시 보호를 위해 어떤 글자 키를 눌렀는지(Key Identity)를 배제하고 순수 타이밍 파형만 입력받습니다. 향후에는 실제 문자 대신 **키보드 물리 좌표 간 손가락 이동 거리(`Δx, Δy`)와 양손 교차 여부(Hand Alternation Flag)**만 비식별 메타 채널로 추가하여 자유 문장(Free-Text) 교차 식별 정확도를 한 단계 더 끌어올릴 계획입니다.
3. **[서비스 차원] 점진적 템플릿 적응(`Sliding Centroid Adaptation`) 및 사내 SSO/LMS 플러그인화**: 정당 사용자의 로그인이 높은 신뢰도로 승인될 때마다 등록 센트로이드를 지수이동평균(`c_u ← 0.95·c_u + 0.05·z`)으로 미세 갱신하여 수개월간의 타자 숙련도 변화를 자동 추적하고, 온라인 코딩 테스트·LMS 대리 시험 부정행위 실시간 탐지 위젯으로 확장할 수 있습니다.